

Leçon : Introduction au JavaScript

Compétences visées :

- Énoncer les limites du HTML liées à l'interactivité
- Définir JavaScript
- Énoncer les avantages et limites de JavaScript
- Utiliser la balise script

I. DÉFINITION, IMPORTANCE ET LIMITE DE JAVASCRIPT

1. Définition

Un **script** est une suite d'instructions permettant d'automatiser certaines tâches.

Le **JavaScript** est un langage de script orienté objet exécutable par les navigateurs web. Pour cela on dit que c'est un langage qui s'exécute coter client.

2 Importance

JavaScript permet aux programmeurs web de créer les pages dynamiques ; c'est-à-dire :

- Interagir avec l'utilisateur : JavaScript permet au navigateur de traiter directement
- Embellir les pages : Afficher/masquer du texte, faire défiler des objets, créer des info-bulles, gérer des menus ;
- Effectuer des calculs et des tests
- Réagir aux événements de la souris.

3 Limite

Les principales limites de ce langage sont : ne permet pas la connexion à une base de données, ne permet pas de lire ou écrire dans un disque dur, n'est pas autonome car ne peut fonctionner sans le HTML, est très dépendant du navigateur.

II. UTILISATION DE LA BALISE SCRIPT

Le code JavaScript est placé entre les balises : selon l'une des structures suivantes :

<code><script type="text/javascript"></code> taper des instructions JavaScript ici <code></script></code>	<code><script language="javascript"></code> taper des instructions JavaScript ici <code></script></code>
---	--

Notons qu'il existe plusieurs façons d'utiliser un script dans une page web :

- De manière interne : dans l'entête ou le corps de la pageDe manière externe : dans un fichier externe (avec une extension .js) à la page web.

LECON : Notions de base du JavaScript

Compétences visées :

- Déclarer les variables en JavaScript
- Effectuer les opérations d'entrées/sorties en JavaScript
- Effectuer les opérations d'entrées/sorties en JavaScript
- Utiliser les opérateurs en JavaScript

I. DÉCLARATION DES VARIABLES EN JAVASCRIPT

En JavaScript, les noms de variable peuvent être aussi longs que l'on désire, mais doivent répondre à certains critères :

- Commencer par une lettre (majuscule ou minuscule) ou un "_" ;
- Comporter uniquement des lettres, des chiffres et les caractères _ et \$;
- Ne peuvent pas être des mots réservés du langage.

Syntaxe : `var nom_variable = valeur`

Exemples : `var nombre = 19` `var prenom = "zavier"` `var note`

Le mot clé de déclaration d'une constante est « const ». Syntaxe : `Const nom_constante = valeur`

Exemples : `Const taux_TVA = 0.1925` `Const PI = 3.14`

II. LES INSTRUCTIONS DE BASES EN JAVASCRIPT

1) L'instruction d'affectation.

Cette instruction permet d'attribuer une valeur ou le résultat d'une expression à une variable.

Syntaxe : `nom_variable=valeur ;` Exemple : `nombre = 19` `prenom = "zavier"`

2 L'instruction de lecture ou d'entrée.

Cette instruction permet de récupérer les informations saisies par l'utilisateur et de les stocker dans les variables. Pour cela on utilisera la méthode `prompt()`.

Syntaxe : `variable=prompt("texte à afficher", "valeur par défaut")`. La valeur par défaut est optionnelle.

`Nom = prompt("Entrer votre nom" , "Bineta") ;`

3 L'instruction d'affichage ou de sortie ou d'écriture.

Pour afficher du texte en JavaScript on peut utiliser la méthode `write()` ou la méthode `alert()`

`document.write("texte à afficher");` ou `alert("texte à afficher");`
`document.write(nom_variable);` ou `alert(nom_variable);`

Remarque : La réponse de la commande prompt() est toujours une chaîne de caractères.

Si on désire récupérer un nombre, il faut utiliser la fonction parseInt (pour convertir en un nombre entier) ou parseFloat (pour la convertir en nombre à virgule). Syntaxe : parseInt(chaine) ou parseInt(var) ; parseFloat(chaine) ou parseFloat(var)

Exemples :

```
Age= prompt("Entrer votre age");      Ou      Age = parseInt(prompt("Entrer votre ag  
parseInt(Age);  
parseInt("23AZer") et parseInt("23"); //ces deux instructions retournent le  
nombre 23.  
Rayon = parseFloat("24.568Bonjour38"); //cette instruction renvoie le nombre  
24.568
```

III. LES OPÉRATIONS EN JAVASCRIPT

1) Les opérateurs arithmétiques

Opérateur	Description
+	Addition
-	Soustraction
*	Multiplification
/	Division
^	Puissance
%	Modulo (a%b renvoie le reste de la division entière de a par b)
++	Incrément // x++ équivaut à x = x + 1
--	Décrément // x-- équivaut à x = x - 1

2) Les opérateurs logiques

Opérateur	Description
&&	ET logique
	OU logique
!	Négation logique

3) Les opérateurs de comparaison

Opérateur	Description
==	Est égal à
!=	Différent de
>	Strictement supérieur à
<	Strictement inférieur à
>=	Supérieur ou égal
<=	Inférieur ou égal

4) L'opérateur de chaîne ou de concaténation

L'opération entre les chaînes de caractère s'appelle la concaténation, c'est-à-dire la juxtaposition des objets de type chaîne. L'opérateur de concaténation est '+'.

Exemples : a=2 ; b=8 ; c=5 ; texte1= "bonjour " ; texte2= "le monde ! " ;

c = b - c ; → 3 d = (a > 0) && (c < b) ; → False (0)

LECON : Structures de contrôles, tableaux et fonctions en JavaScript

Compétences visées :

- Traduire des algorithmes séquentiels, conditionnels et itératifs en JavaScript
- Déclarer un tableau et une fonction en JavaScript

I. LES STRUCTURES CONDITIONNELLES

a) Structure avec une alternative (if...)

La syntaxe est la suivante :

```
If (condition) {  
// liste d'instructions  
}
```

b) Structure avec deux alternatives (if ... else...)

La syntaxe est la suivante

```
if (condition) {  
// listes d'instructions }  
else {  
// autres séries d'instructions  
}
```

II. STRUCTURES RÉPÉTITIVES (OU BOUCLES OU ITÉRATIVES)

a) While

Sa syntaxe est la suivante :

```
while ( ) {
```

```
<action> ;  
... }
```

Exemple :

```
<script language="JavaScript">  
compt=1;  
while (compt<5){  
document.write ("ligne : " + compt + "  
<br>");  
compt++;  
}  
document.write("fin de la boucle");  
</script>
```

b) Do...While

Sa syntaxe est la suivante :

```
do {  
<action>  
...  
} while ( )(<condition>)
```

Exemple :

```
<script language="JavaScript">  
compt=1;  
do{ document.write ("ligne : " + compt + "  
<br>");  
compt++;  
} while (compt<5)  
document.write("fin de la boucle");  
</script>
```

c) For

Elle correspond à la structure algorithmique «Pour» et la syntaxe est la suivante :

```
for (compteur ; condition; modification du compteur) {
```

<action>

...

}

Exemple :

```
< script language="JavaScript">
```

```
var somme=20;
```

```
for (i=10 ;i<=12 ; i++){
```

```
somme+=i ;
```

```
document.write("La somme est : " +somme+"<br>");
```

```
}
```

```
</script>
```

III. 10- TABLEAUX ET FONCTIONS EN JS

a) Les tableaux

De manière interne, les tableaux sont représentés de la manière suivante :

Indice 0	Indice 1	Indice 2	Indice 3	...	Indice n
Donnée 1	Donnée 2	Donnée 3	Donnée 4	...	Indice n

La taille d'un tableau est dynamique. Elle augmente au fur et à mesure qu'on y ajoute des éléments. Elle est donnée par la propriété «length» (exemple : tableau1.length)

syntaxe

```
tableau1 = new Array( ) ;// sans initialisation
```

```
tableau2 = new Array("donnée 1", "donnée 2", "donnée 3", "donnée 4") ;
```

```
tableau3 = ["donnée 1", "donnée 2", "donnée 3", "donnée 4"];
```

Pour accéder à un élément du tableau, on utilise la syntaxe suivante : tableau1 [indice].

b) les fonctions

La déclaration d'une fonction ou procédure se fait en respectant la syntaxe suivante :

```
function nom_fonction(parametre 1, parametre 2, ...) {
```

```
<instruction>
```

```
...
```

```
Return resultat
```

```
}
```

Exemple : fonction calculant le carrée d'un nombre qu'on lui passe en paramètre.

Exemple : fonction calculant le carrée d'un nombre qu'on lui passe en paramètre. fonction

```
carre(nombre){
```

```
  res = nombre*nombre ;
```

```
  return res;
```

```
}
```

Remarque : Toute variable déclarée en dehors de toute fonction, est dite **variable globale** et peut donc être exploitée partout dans le script tandis qu'une variable déclarée par le mot clé `var` dans une fonction aura une portée limitée à cette seule fonction et ne pourra donc pas être exploitée ailleurs dans le script. D'où le nom **de variable locale**. Par contre, toujours dans une fonction, si la variable est déclarée de manière implicite (sans utiliser le mot `var`), sa portée sera globale.